Table of Contents

1. Introduction

The Arithmetic Logic Unit (ALU) is a core component of the Central Processing Unit (CPU) responsible for performing arithmetic and logical operations on binary data The accuracy and efficiency of the ALU have a direct impact on the overall system performance

The problem addressed by this project is the need to design a flexible and efficient ALU capable of executing a set of essential operations, including:

- Arithmetic operations (addition, subtraction, multiplication, division)
- Logical operations (AND, OR, XOR, NOT)
- Bit-level data manipulation (shifting, masking)
- Comparison operations for binary values

A well-structured ALU that supports these functions enhances the processor's ability to handle various computational tasks effectively which is the main goal of this project.

2. Description and Explanation

The **Arithmetic Logic Unit (ALU)** is designed to perform arithmetic and logical operations as part of the processor's execution cycle The ALU's design involves key components like multiplexers, full adders, shifters, and Boolean gates.

1. Basic Operations:

- **Addition**: Implemented using a ripple-carry adder or a carry-lookahead adder.
- **Subtraction**: Implemented using two’s complement.
- **Multiplication**: Implemented using shift-and-add algorithms or Booth's algorithm.
- **Division**: Implemented using restoring or non-restoring division algorithms.

2. Logical Operations:

Logic gates like **AND**, **OR**, **NOT**, **XOR** are used to perform bitwise comparisons of the input operands

3. Flags:

Status flags are generated after each operation:

- **Zero Flag (Z)**: Set if the result is zero.
- **Carry Flag (C)**: Set to detect overflows in unsigned arithmetic.

These flags are stored in a status register and are used to control decision-making in program flow.

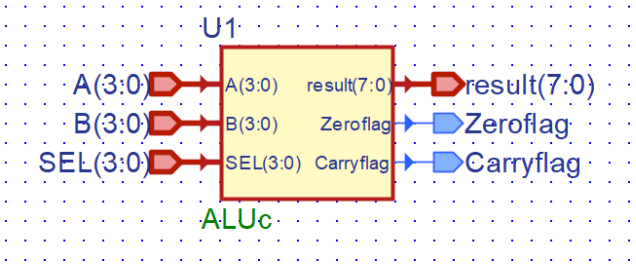
3. Source Code

```
module ALUc (A, B, SEL, result, Zeroflag, Carryflag);

input [3:0] A, B;
input [3:0] SEL;
output reg [7:0] result;
output reg Zeroflag;
output reg Carryflag;

reg [4:0] temp;

always @ (A or B or SEL) begin
    Carryflag = 0;
    case (SEL)
        4'b0000: begin  // Addition
            temp = A + B;
            result = {3'b000, temp[3:0]};
            Carryflag = temp[4];
        end
        4'b0001: begin  // Subtraction
            temp = A - B;
            result = {3'b000, temp[3:0]};
            Carryflag = temp[4];
        end
        4'b0010: result = A * B;  //Multiplication
        4'b0011: begin
            if (B != 0)
                result = A / B; //Division
            else
                result = 8'b00000000;
        end
        4'b0100: result = {4'b0000, A & B}; //AND
        4'b0101: result = {4'b0000, A | B}; //OR
        4'b0110: result = {4'b0000, ~A};    //NOT A
        4'b0111: result = {4'b0000, A ^ B}; //XOR
        4'b1000: result = {4'b0000, ~B};   //NOT B
        default: result = 8'b00000000;
    endcase

    if (result == 8'b00000000)
        Zeroflag = 1'b1;
    else
        Zeroflag = 1'b0;
end

endmodule
```

I

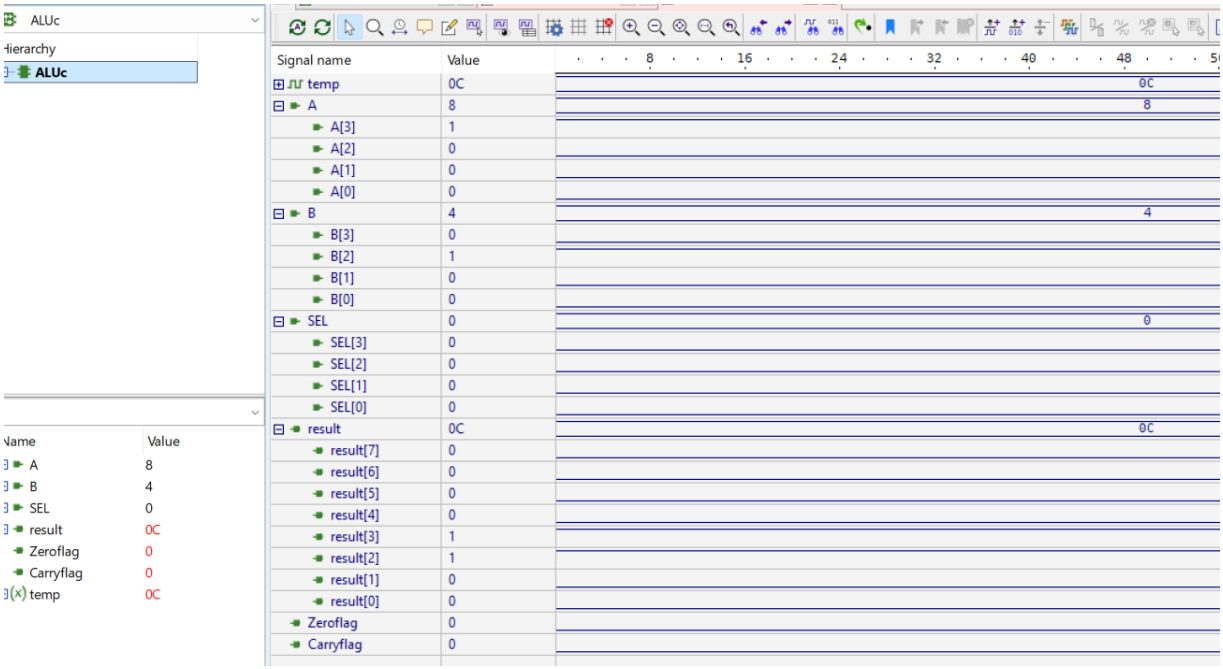
4. Screenshots

4.1. The Diagram

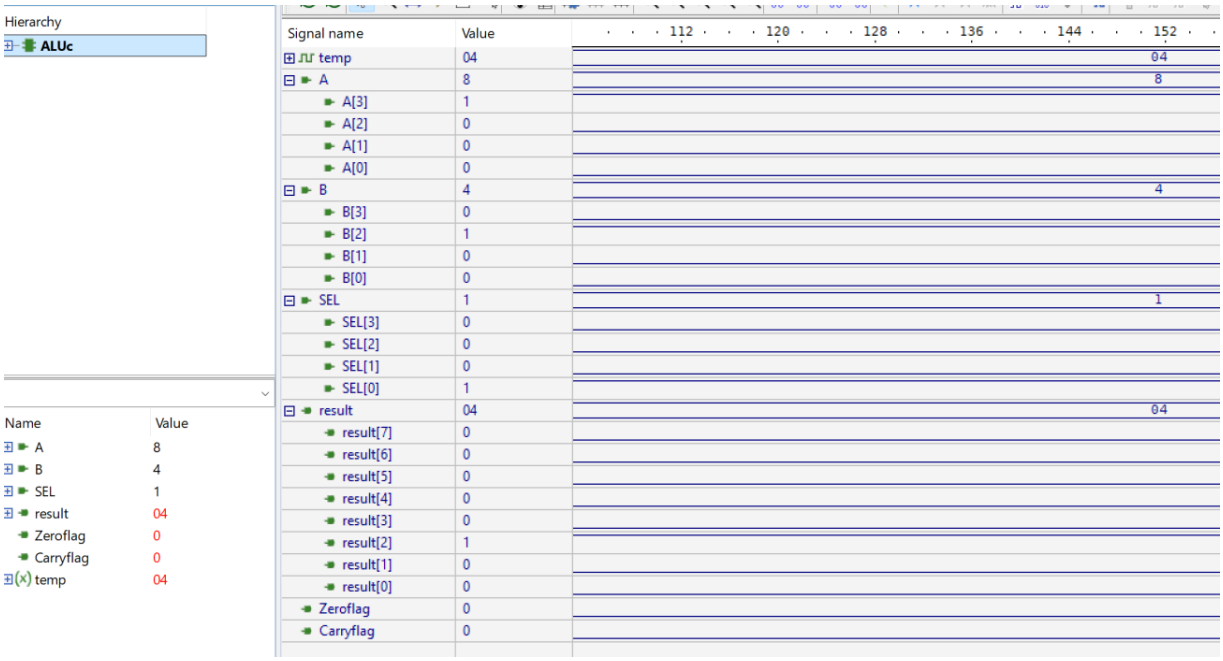


4.2. Simulation Trace

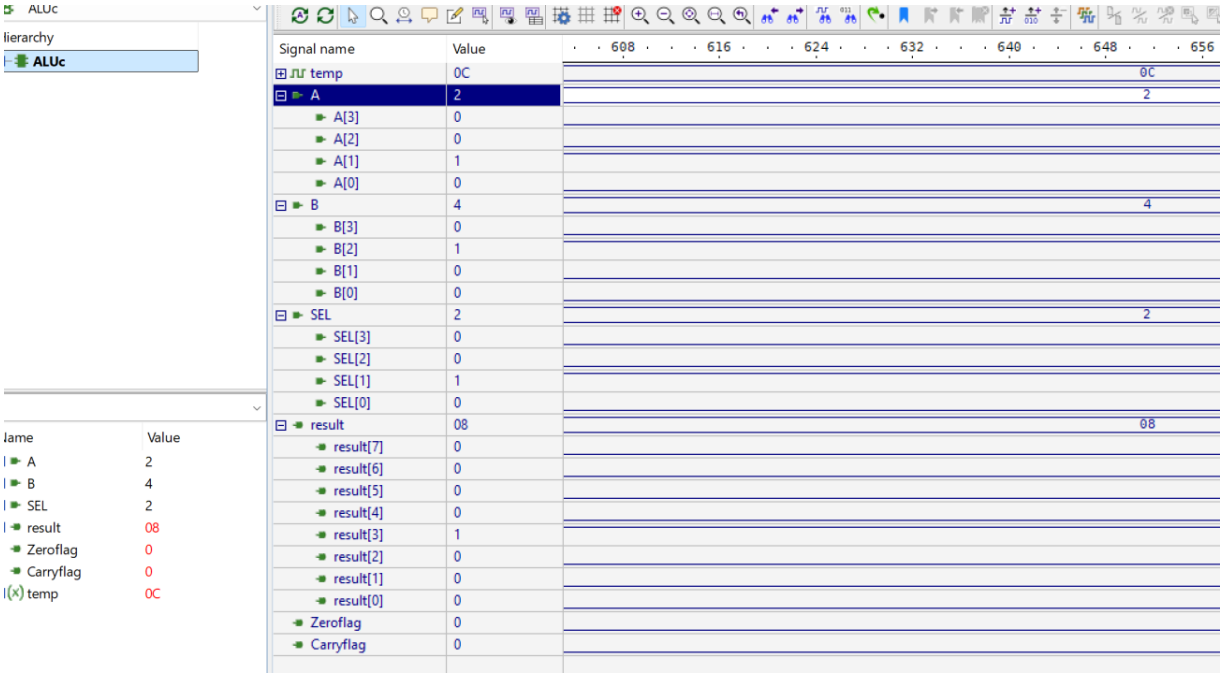
1. Basic Operations: **Addition**:



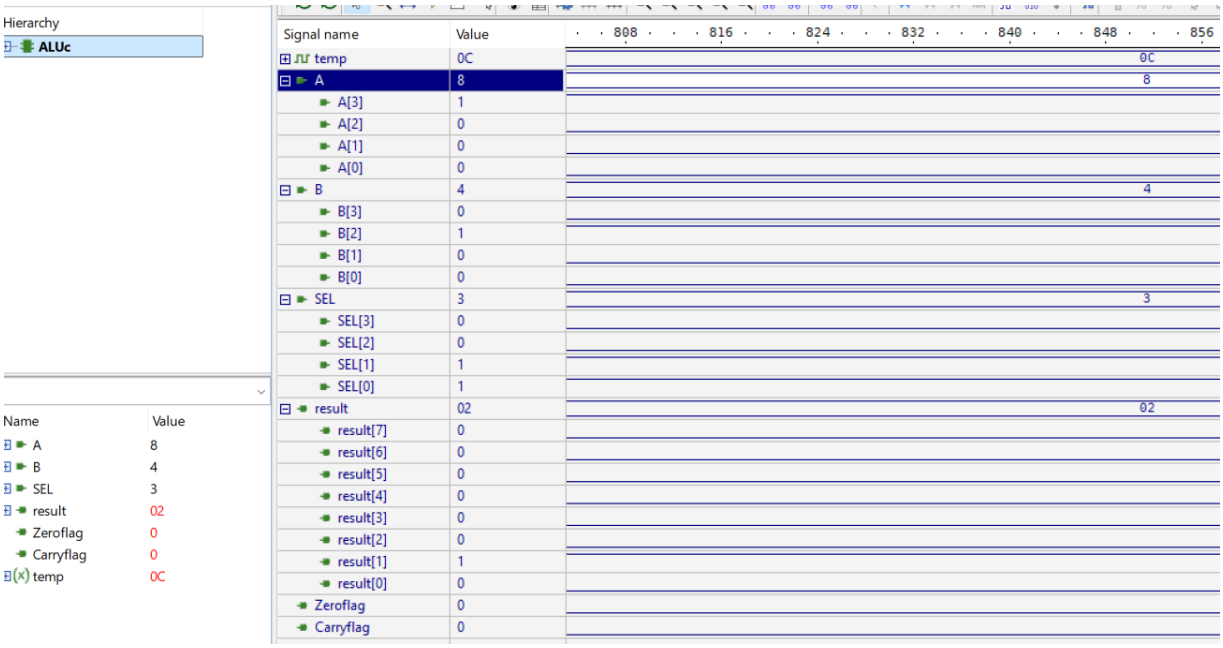
Subtraction:



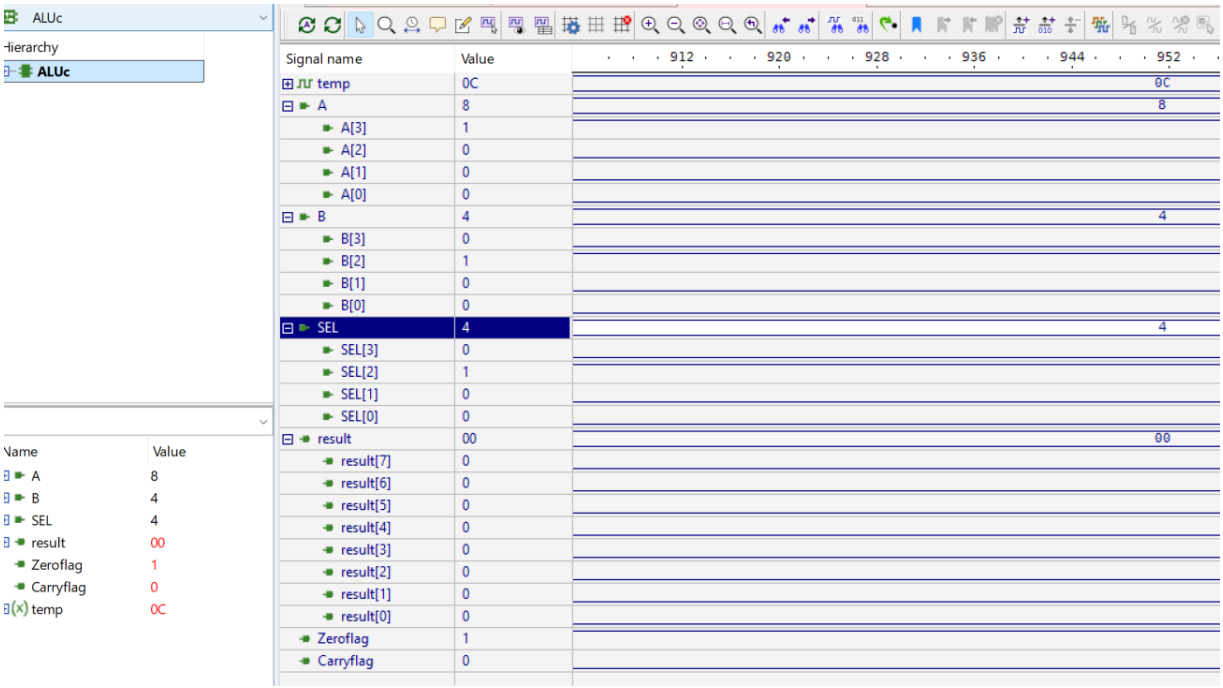
Multiplication:



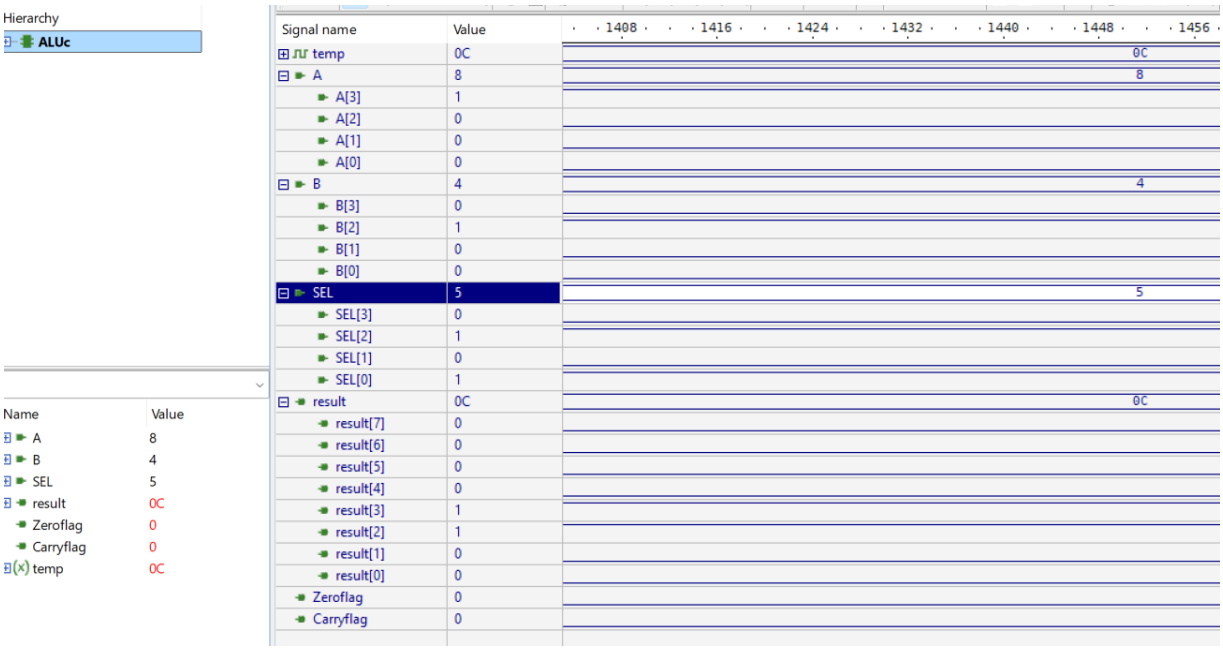
Division:



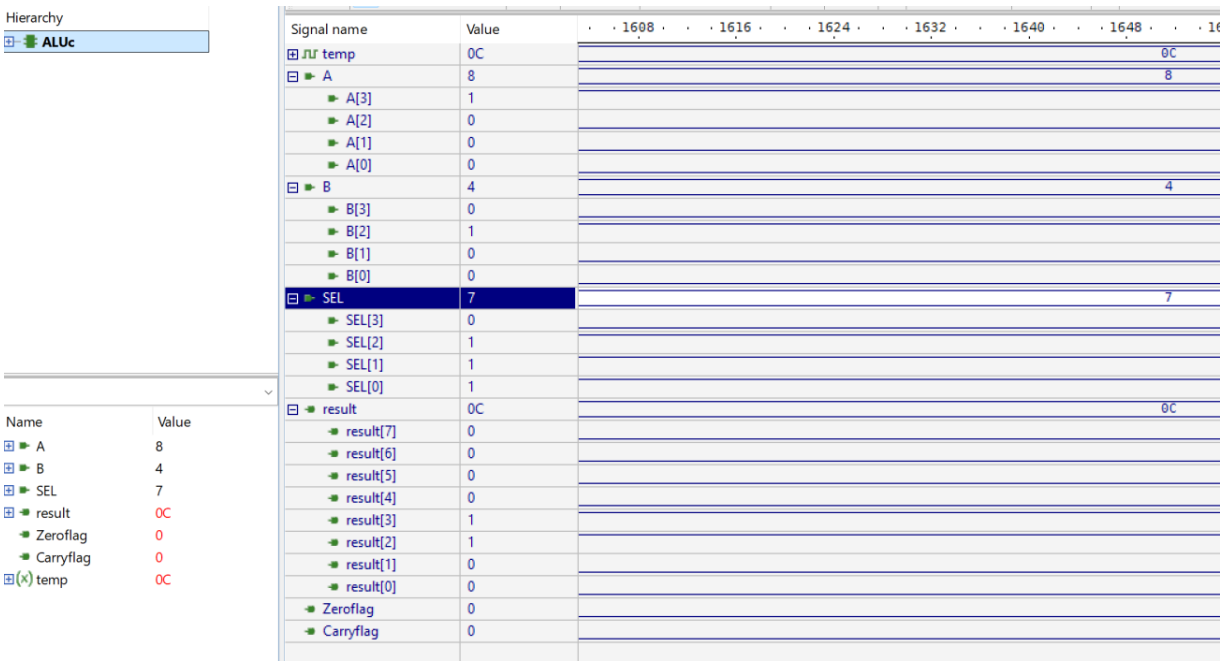
2. Logical Operations: **AND**



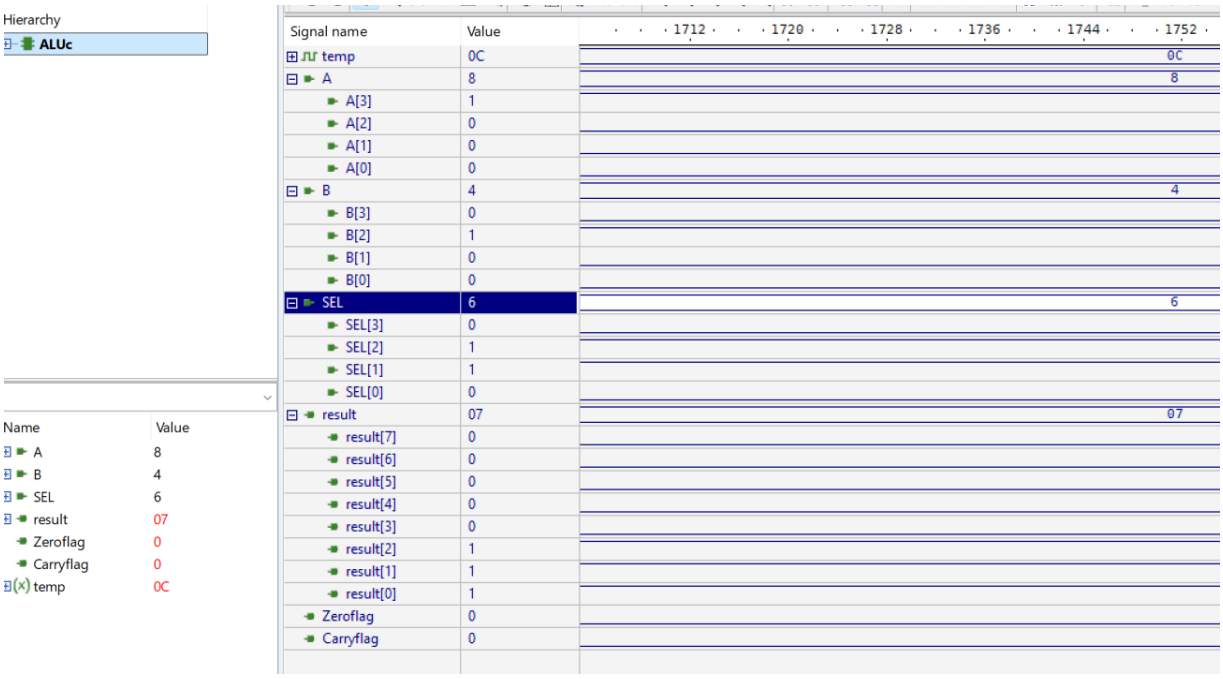
OR:


Hierarchy
ALUc
Name
Value
A
8
B
4
SEL
5
result
0C
Zeroflag
0
Carryflag
0
temp
0C
Signal name
Value
1408
1416
1424
1432
1440
1448
1456
temp
0C
A
8
A[3]
1
A[2]
0
A[1]
0
A[0]
0
B
4
B[3]
0
B[2]
1
B[1]
0
B[0]
0
SEL
5
SEL[3]
0
SEL[2]
1
SEL[1]
0
SEL[0]
1
result
0C
result[7]
0
result[6]
0
result[5]
0
result[4]
0
result[3]
1
result[2]
1
result[1]
0
result[0]
0
Zeroflag
0
Carryflag
0

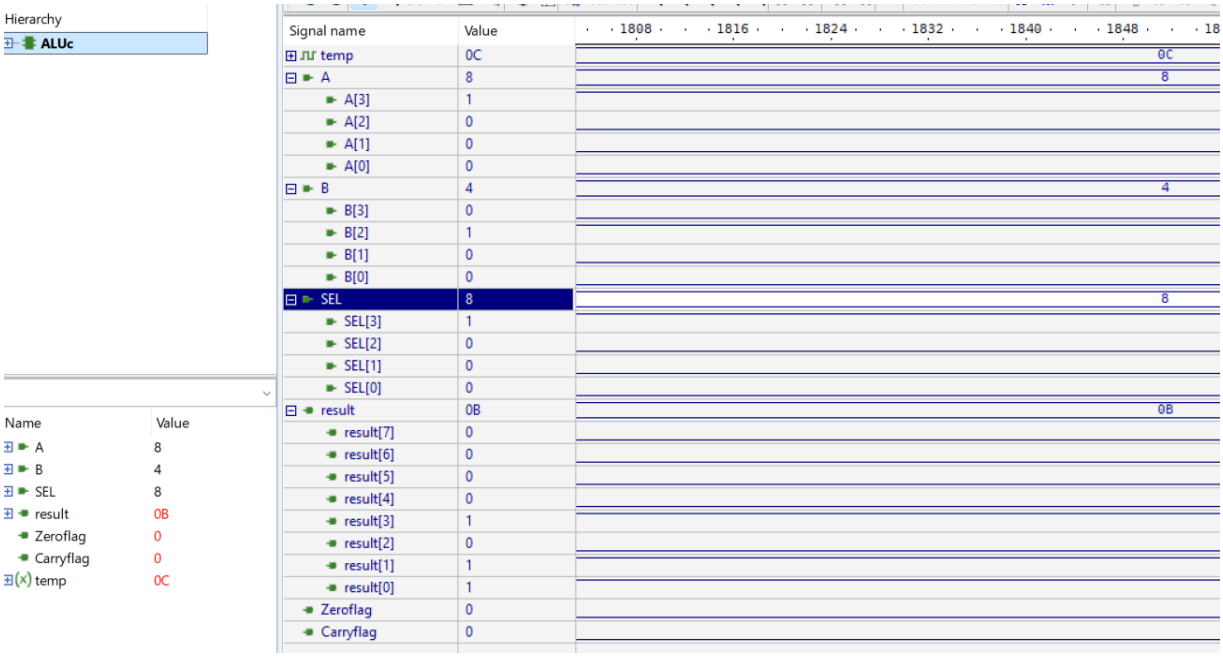
XOR:


Hierarchy
ALUc
Name
Value
A
8
B
4
SEL
7
result
0C
Zeroflag
0
Carryflag
0
temp
0C
Signal name
Value
1608
1616
1624
1632
1640
1648
temp
0C
A
8
A[3]
1
A[2]
0
A[1]
0
A[0]
0
B
4
B[3]
0
B[2]
1
B[1]
0
B[0]
0
SEL
7
SEL[3]
0
SEL[2]
1
SEL[1]
1
SEL[0]
1
result
0C
result[7]
0
result[6]
0
result[5]
0
result[4]
0
result[3]
1
result[2]
1
result[1]
0
result[0]
0
Zeroflag
0
Carryflag
0

NOT A:


Hierarchy
ALUc
Signal name
Value
1712
1720
1728
1736
1744
1752
temp 0C
A 8
A[3] 1
A[2] 0
A[1] 0
A[0] 0
B 4
B[3] 0
B[2] 1
B[1] 0
B[0] 0
SEL 6
SEL[3] 0
SEL[2] 1
SEL[1] 1
SEL[0] 0
result 07
result[7] 0
result[6] 0
result[5] 0
result[4] 0
result[3] 0
result[2] 1
result[1] 1
result[0] 1
Zeroflag 0
Carryflag 0
Name
Value
A 8
B 4
SEL 6
result 07
Zeroflag 0
Carryflag 0
temp 0C

NOT B:


Hierarchy
ALUc
Signal name
Value
1808
1816
1824
1832
1840
1848
temp 0C
A 8
A[3] 1
A[2] 0
A[1] 0
A[0] 0
B 4
B[3] 0
B[2] 1
B[1] 0
B[0] 0
SEL 8
SEL[3] 1
SEL[2] 0
SEL[1] 0
SEL[0] 0
result 0B
result[7] 0
result[6] 0
result[5] 0
result[4] 0
result[3] 1
result[2] 0
result[1] 1
result[0] 1
Zeroflag 0
Carryflag 0
Name
Value
A 8
B 4
SEL 8
result 0B
Zeroflag 0
Carryflag 0
temp 0C

6. References

https://github.com/SarwanShah/8-bit-ALU-Using-Logic-Gates-2017

https://github.com/Chandutv0808/-ARITHMETIC-LOGIC-UNIT-ALU-

https://www.slideshare.net/slideshow/vlsi-design-final-project-32-bit-alu/64913691

https://engr210.github.io/projects/P6.html